

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 1 – Test Case Design Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 1 – Test Case Design Assignment
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	Y. Ramadevi	Reg. No.:	192373065
Date:	21/08/2026	Duration:	60 minutes

Code of Conduct

I Y. Ramadevi - 192373065 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student **Y. Ramadevi**

Annexure A – Test Case Design Assignment

Containerized CI/CD Pipeline for Automated Application Deployment

1. Introduction

Modern software applications require fast, reliable, and automated mechanisms for building, testing, and deploying applications. A Containerized CI/CD Pipeline for Automated Application Deployment integrates source code management, automated testing, Docker containerization, Continuous Integration (CI), Continuous Deployment (CD), and deployment strategies to ensure that software changes can be delivered efficiently and safely.

In the proposed system, developers push application code to a Git repository. The CI/CD platform automatically detects the changes and initiates a pipeline. The pipeline performs code checkout, dependency installation, automated testing, application building, Docker image creation, container registry publishing, and deployment to the target environment.

The system uses technologies such as:

- Git/GitHub for source code management
- Jenkins or GitHub Actions for CI/CD automation
- Docker for application containerization
- Automated testing tools for validating application functionality
- Docker Registry for storing container images
- Blue-Green Deployment or Rolling Update for controlled production releases
- Monitoring and logging mechanisms for deployment verification

The purpose of this assignment is to identify the functional and non-functional requirements of the proposed CI/CD system and design comprehensive test cases covering normal, boundary, invalid, and exceptional conditions.

2. Problem Statement

Containerized CI/CD Pipeline for Automated Application Deployment

Design and test an automated CI/CD system that performs the following operations:

1. Detects changes pushed to the source code repository.
 2. Triggers the CI/CD pipeline automatically.
 3. Checks out the latest application source code.
 4. Installs required dependencies.
 5. Builds the application.
 6. Executes automated unit and integration tests.
 7. Stops the pipeline when critical tests fail.
 8. Builds a Docker image after successful validation.
 9. Tags the Docker image with an appropriate version.
 10. Pushes the Docker image to a container registry.
 11. Deploys the image to the target environment.
 12. Verifies deployment health.
 13. Performs rollback when deployment or health verification fails.
 14. Supports deployment strategies such as Blue-Green Deployment and Rolling Updates.
 15. Generates pipeline logs and execution status for troubleshooting.
-

3. Objectives

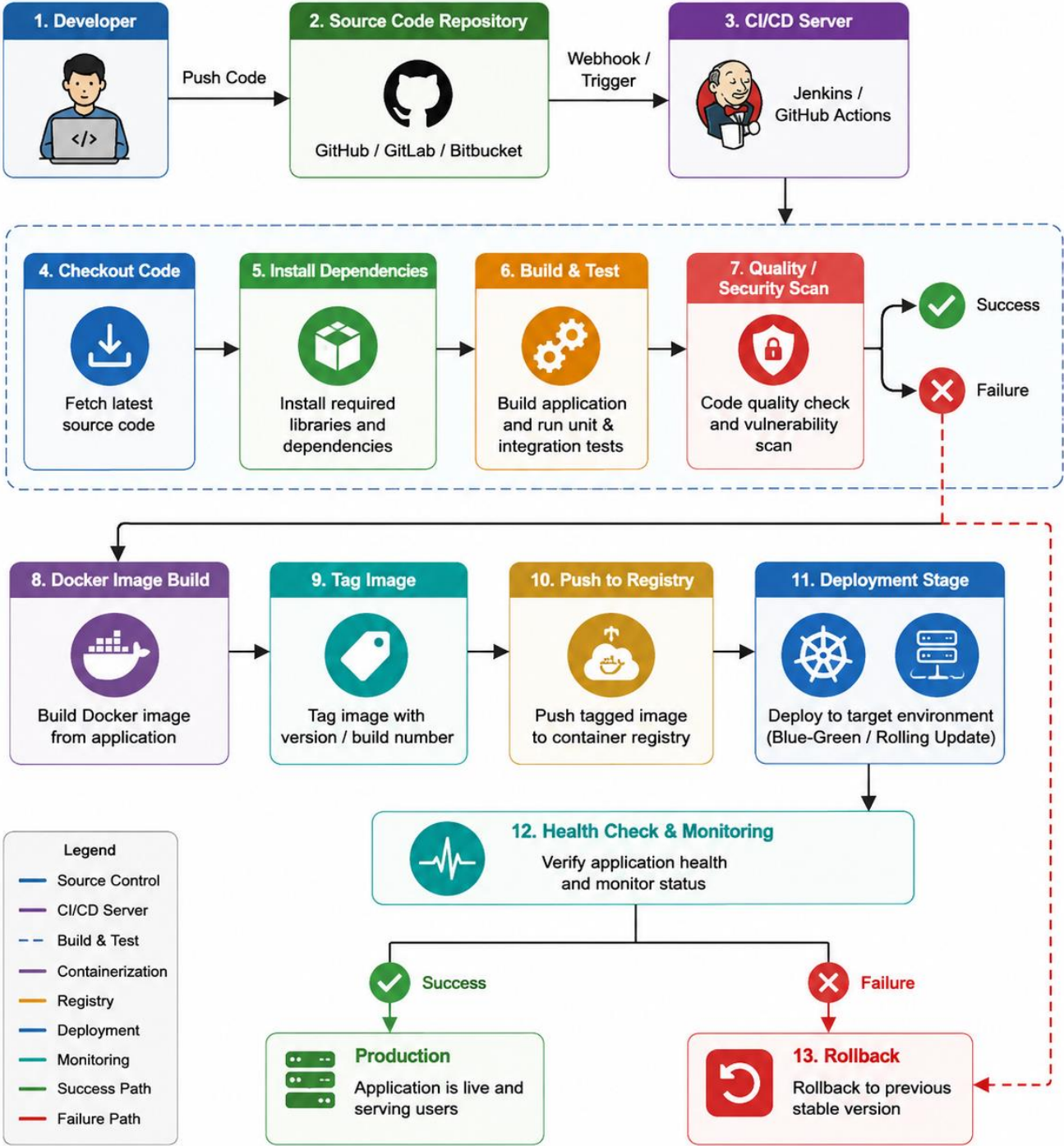
The objectives of the test case design are:

- To verify the correctness of CI/CD pipeline execution.
 - To validate automatic pipeline triggering.
 - To ensure successful application building and testing.
 - To verify Docker image generation and container registry operations.
 - To test deployment functionality under normal and failure conditions.
 - To validate rollback and recovery mechanisms.
 - To test Blue-Green and Rolling Update deployment strategies.
 - To verify pipeline reliability, security, performance, and usability.
 - To ensure traceability between requirements, test scenarios, and test cases.
-

4. System Overview

The proposed CI/CD workflow is shown below:

Containerized CI/CD Pipeline for Automated Application Deployment



5. Functional Requirements

Requirement ID	Functional Requirement
----------------	------------------------

FR-01	The system shall detect a valid code push to the configured repository.
FR-02	The system shall automatically trigger the CI/CD pipeline after a repository event.
FR-03	The pipeline shall check out the latest source code.
FR-04	The pipeline shall install required application dependencies.
FR-05	The system shall build the application successfully when the source code is valid.
FR-06	The system shall execute automated unit and integration tests.
FR-07	The pipeline shall stop subsequent stages when critical tests fail.
FR-08	The system shall build a Docker image after successful testing.
FR-09	The Docker image shall be tagged with a valid version or build number.
FR-10	The system shall push the generated image to the configured container registry.
FR-11	The system shall deploy the approved Docker image to the target environment.
FR-12	The system shall perform application health checks after deployment.
FR-13	The system shall automatically roll back when deployment or health verification fails.
FR-14	The system shall support Blue-Green deployment.
FR-15	The system shall support Rolling Update deployment.
FR-16	The system shall display pipeline execution status and logs.
FR-17	The system shall prevent deployment when required credentials are invalid or unavailable.

6. Non-Functional Requirements

Requirement ID	Non-Functional Requirement
NFR-01	Pipeline execution shall complete within the defined acceptable time limit for a normal build.
NFR-02	The system shall provide reliable execution and recover safely from failures.
NFR-03	Sensitive credentials shall not be exposed in pipeline logs.
NFR-04	Unauthorized users shall not be allowed to trigger protected production deployments.
NFR-05	Pipeline logs shall provide sufficient information for troubleshooting.
NFR-06	The deployment process shall maintain application availability during supported zero-downtime deployments.

Requirement ID	Non-Functional Requirement
NFR-07	The system shall handle multiple pipeline requests according to the configured concurrency policy.
NFR-08	The pipeline interface and status indicators shall be understandable to authorized users.

7. Test Scenarios

Scenario ID	Test Scenario	Requirement Coverage
TS-01	Trigger pipeline after valid code push	FR-01, FR-02
TS-02	Reject or ignore invalid/unconfigured repository event	FR-01
TS-03	Verify source code checkout	FR-03
TS-04	Verify dependency installation	FR-04
TS-05	Verify successful application build	FR-05
TS-06	Verify behavior when application build fails	FR-05
TS-07	Verify successful automated test execution	FR-06
TS-08	Verify pipeline termination when critical tests fail	FR-07
TS-09	Verify Docker image generation	FR-08
TS-10	Verify Docker image tagging	FR-09
TS-11	Verify successful image push to registry	FR-10
TS-12	Verify behavior for invalid registry credentials	FR-10, FR-17
TS-13	Verify successful application deployment	FR-11
TS-14	Verify deployment health check	FR-12
TS-15	Verify rollback after failed deployment	FR-13
TS-16	Verify Blue-Green deployment switching	FR-14
TS-17	Verify Rolling Update behavior	FR-15
TS-18	Verify pipeline logs and status reporting	FR-16
TS-19	Verify credential masking and security	NFR-03
TS-20	Verify unauthorized deployment prevention	NFR-04
TS-21	Verify pipeline execution performance	NFR-01
TS-22	Verify concurrent pipeline request handling	NFR-07
TS-23	Verify usability of pipeline status information	NFR-08
TS-24	Verify recovery from infrastructure or network failure	NFR-02

8. Detailed Test Cases

Test Case 1: Valid Code Push Triggers Pipeline

Field	Description
Test Case ID	TC-CICD-001
Test Scenario	Automatic pipeline triggering
Test Type	Normal

Field	Description
Preconditions	Repository and webhook are configured correctly
Test Steps	1. Modify valid source code. 2. Commit the changes. 3. Push the code to the configured branch. 4. Observe pipeline execution.
Test Data	Valid commit pushed to main branch
Expected Result	Pipeline is automatically triggered and begins execution for the latest commit.
Actual Result	To be filled during execution
Pass/Fail Status	To be filled during execution

Test Case 2: Pipeline Trigger with Unconfigured Repository

Field	Description
Test Case ID	TC-CICD-002
Test Scenario	Invalid repository event
Test Type	Invalid
Preconditions	Repository is not configured as an approved pipeline source
Test Steps	1. Push code to an unconfigured repository. 2. Monitor the CI/CD server.
Test Data	Commit from unauthorized repository
Expected Result	No production pipeline is triggered. The event is ignored or rejected according to configuration.
Actual Result	To be filled during execution
Pass/Fail Status	To be filled during execution

Test Case 3: Successful Source Code Checkout

Field	Description
Test Case ID	TC-CICD-003
Test Scenario	Source code checkout
Test Type	Normal
Preconditions	Pipeline is triggered and repository access is valid
Test Steps	1. Trigger pipeline. 2. Execute checkout stage. 3. Verify commit version.
Test Data	Valid repository URL and commit ID
Expected Result	Latest required source code is downloaded successfully. The checked-out commit matches the triggered revision.
Actual Result	To be filled during execution
Pass/Fail Status	To be filled during execution

9. Application of Test Design Techniques

9.1 Equivalence Partitioning

Equivalence Partitioning divides input conditions into groups that are expected to behave similarly.

Example: Registry Credentials

Partition	Test Data	Expected Result
Valid partition	Valid username and token	Authentication succeeds
Invalid partition	Incorrect username or token	Authentication fails
Empty partition	Blank credentials	Validation or authentication error
Expired partition	Expired token	Authentication fails securely

The representative test cases are:

- TC-CICD-015 – Valid registry credentials
- TC-CICD-016 – Invalid registry credentials
- TC-CICD-027 – Credential security verification

9.2 Boundary Value Analysis

Boundary Value Analysis is applied to inputs where minimum and maximum limits may cause failures.

Example: Docker Image Tag

Input Condition	Test Value Type	Expected Result
Minimum supported length	Lower boundary	Accepted
One value above minimum	Valid	Accepted
Maximum supported length	Upper boundary	Accepted
One value above maximum	Invalid	Rejected

This technique is covered by:

TC-CICD-014 – Boundary Test for Image Tag Length

Boundary analysis can also be applied to:

- Maximum pipeline timeout
 - Maximum concurrent builds
 - Minimum healthy replicas
 - Maximum retry attempts
-

9.3 Decision Table Testing

Decision Table Testing is useful when pipeline actions depend on multiple conditions.

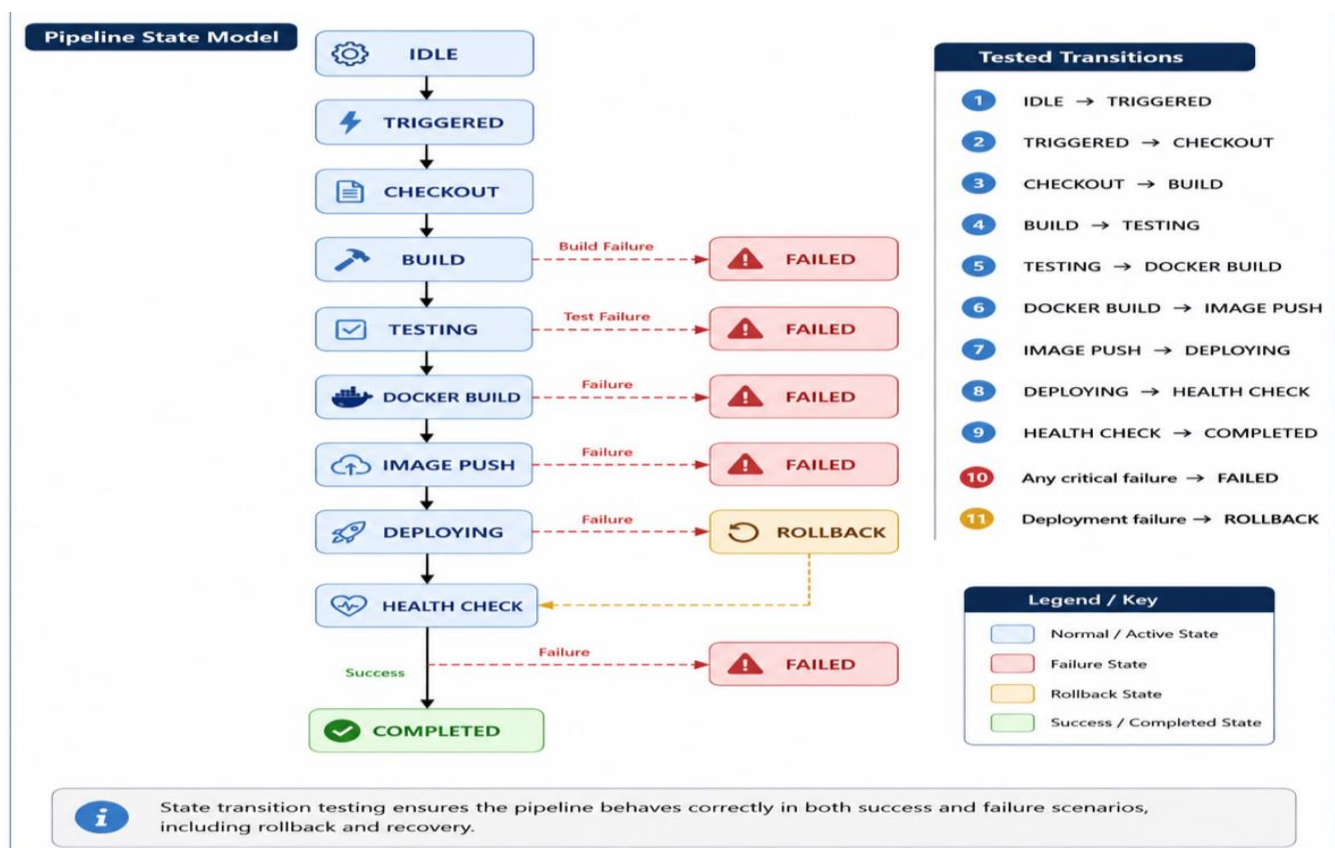
Deployment Decision Table

Build Successful	Tests Passed	Image Push Successful	Health Check Passed	Expected Action
Yes	Yes	Yes	Yes	Deploy and mark success
No	-	-	-	Stop pipeline
Yes	No	-	-	Stop pipeline
Yes	Yes	No	-	Stop pipeline and report registry failure
Yes	Yes	Yes	No	Roll back deployment

The decision table is verified through:

- TC-CICD-008
- TC-CICD-010
- TC-CICD-016
- TC-CICD-020

9.4 State Transition Testing



10. Requirement Traceability Matrix

Requirement ID	Requirement	Test Scenario	Test Case IDs
FR-01	Detect repository event	TS-01, TS-02	TC-001, TC-002
FR-02	Trigger pipeline automatically	TS-01	TC-001
FR-03	Checkout source code	TS-03	TC-003, TC-004
FR-04	Install dependencies	TS-04	TC-005, TC-006
FR-05	Build application	TS-05, TS-06	TC-007, TC-008
FR-06	Execute automated tests	TS-07	TC-009
FR-07	Stop after critical test failure	TS-08	TC-010
FR-08	Build Docker image	TS-09	TC-011, TC-012
FR-09	Tag Docker image	TS-10	TC-013, TC-014
FR-10	Push image to registry	TS-11, TS-12	TC-015, TC-016
FR-11	Deploy application	TS-13	TC-017, TC-018
FR-12	Perform health checks	TS-14	TC-019
FR-13	Automatic rollback	TS-15	TC-020
FR-14	Blue-Green deployment	TS-16	TC-021, TC-022
FR-15	Rolling update	TS-17	TC-023, TC-024
FR-16	Logs and execution status	TS-18	TC-025
FR-17	Prevent deployment with invalid credentials	TS-12	TC-016
NFR-01	Performance	TS-21	TC-028
NFR-02	Reliability and recovery	TS-24	TC-020, TC-024, TC-030
NFR-03	Credential security	TS-19	TC-016, TC-027
NFR-04	Authorization	TS-20	TC-026
NFR-05	Troubleshooting logs	TS-18	TC-025
NFR-06	Deployment availability	TS-16, TS-17	TC-021, TC-023
NFR-07	Concurrent execution	TS-22	TC-029
NFR-08	Usability	TS-23	TC-025

11. Test Coverage Summary

The proposed test suite provides coverage across multiple testing categories.

Test Category	Coverage
Normal Functional Testing	Repository trigger, checkout, build, testing, image creation, push, deployment

Boundary Testing	Image tag validation, concurrent requests, configured limits
Invalid Testing	Invalid credentials, invalid repository, invalid dependencies, invalid Dockerfile
Exceptional Testing	Build failure, test failure, network interruption, deployment failure
Security Testing	Credential masking and unauthorized deployment prevention
Reliability Testing	Rollback, recovery, rolling update failure, network failure
Performance Testing	Pipeline execution time
Deployment Testing	Blue-Green deployment and Rolling Updates
Logging Testing	Pipeline execution logs and error reporting
State Transition Testing	Pipeline state progression and failure transitions

Coverage Status

- Functional Requirements Covered: 17 / 17
- Non-Functional Requirements Covered: 8 / 8
- Total Detailed Test Cases: 30
- Normal Conditions: Covered
- Boundary Conditions: Covered
- Invalid Conditions: Covered
- Exceptional Conditions: Covered
- Equivalence Partitioning: Applied
- Boundary Value Analysis: Applied
- Decision Table Testing: Applied
- State Transition Testing: Applied

12. Sample Test Execution Summary

The following table can be completed after executing the test cases.

Test Case ID	Execution Result	Status
TC-CICD-001	Pipeline triggered successfully	Pass/Fail
TC-CICD-005	Dependencies installed	Pass/Fail
TC-CICD-007	Application build completed	Pass/Fail
TC-CICD-009	Automated tests executed	Pass/Fail
TC-CICD-011	Docker image created	Pass/Fail

TC-CICD-015	Image pushed to registry	Pass/Fail
TC-CICD-017	Application deployed	Pass/Fail
TC-CICD-019	Health check completed	Pass/Fail
TC-CICD-020	Rollback validated	Pass/Fail
TC-CICD-021	Blue-Green deployment validated	Pass/Fail
TC-CICD-023	Rolling update validated	Pass/Fail
TC-CICD-027	Credentials masked in logs	Pass/Fail
TC-CICD-028	Performance requirement verified	Pass/Fail
TC-CICD-030	Network failure handled	Pass/Fail

13. Conclusion

The Containerized CI/CD Pipeline for Automated Application Deployment was analyzed from a quality assurance and testing perspective. Functional and non-functional requirements were identified and mapped to comprehensive test scenarios and detailed test cases.

The test suite covers the complete CI/CD lifecycle, including code triggering, source checkout, dependency installation, application build, automated testing, Docker image generation, image tagging, registry operations, application deployment, health verification, rollback, Blue-Green deployment, Rolling Updates, security, logging, performance, and failure recovery.

A total of 30 comprehensive test cases were designed to cover normal, boundary, invalid, and exceptional conditions. Multiple test design techniques, including Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, and State Transition Testing, were applied to improve the quality and completeness of the test design.

The Requirement Traceability Matrix demonstrates strong mapping between requirements, scenarios, and test cases, ensuring high functional coverage. This test case design provides a structured approach for verifying that the proposed containerized CI/CD system is correct, reliable, secure, maintainable, and suitable for automated production deployment.